

HW1: Image Classification Report

Visual Recognition using Deep Learning, NYCU

朱驛庭, 111550093

1. Introduction

Fine-grained image classification remains a challenging task in computer vision, as it requires the model to distinguish between visually similar categories that share common structures but differ in subtle details [2]. In this work, we address a 100-class image classification problem spanning diverse fine-grained categories including insects, plants, and birds. The dataset comprises 20,724 training images, 300 validation images, and 2,344 test images, presenting challenges such as complex backgrounds, intra-class variation, and the presence of multiple objects within a single scene.

We adopt a transfer learning approach by fine-tuning a ResNeSt-200 [14] backbone pretrained on ImageNet [1], leveraging its split-attention mechanism for improved feature representation. Our training pipeline incorporates an extensive augmentation strategy combining TrivialAugmentWide [6], CutMix [12], MixUp [13], and Random Erasing [16], together with label smoothing [9] and test-time augmentation (TTA) [3] during inference. We further investigate the effect of very large dropout rates [15] during fine-tuning and compare several learning rate scheduling strategies. The complete implementation, including training configurations and evaluation scripts, is available in the project repository at <https://github.com/ChuEating1005/VRDL/tree/main/HW1>.

The report is organized as follows. Section 2 describes the model architecture, data augmentation pipeline, and hyperparameter configuration. Section 3 presents the training and validation performance as well as competition results. Section 4 provides ablation studies on dropout rates and learning rate schedulers.

2. Method

2.1. Model Architecture

We adopt ResNeSt-200 [14] as our backbone, loaded via the `timm` library [10] with ImageNet-pretrained weights. ResNeSt extends the ResNeXt [11] architecture by introducing a Split-Attention block, which ap-

plies channel-wise attention across feature-map groups (termed *cardinal groups* and *radix splits*). This enables the network to adaptively recalibrate feature responses at each residual block, yielding richer representations for fine-grained recognition without additional computational overhead at inference.

The pretrained classification head is replaced with a new fully connected layer mapping from the backbone’s 2048-dimensional feature space to the 100 target classes. An optional Dropout [8] layer is inserted before this final linear layer, which we explore in Sec. 4.1. The total model has approximately 68M parameters, well under the 100M parameter limit.

2.2. Data Augmentations

Our augmentation pipeline operates at two levels: **per-sample transforms** applied during data loading and **batch-level augmentations** applied after collation.

Per-sample transforms. Each training image undergoes the following sequence:

1. **RandomResizedCrop** to 224×224 with scale range $[0.6, 1.0]$, providing multi-scale views of the input.
2. **RandomHorizontalFlip** with probability 0.5.
3. **TrivialAugmentWide** [6], a tuning-free augmentation policy that uniformly samples a single transform with a random magnitude at each iteration.
4. **Normalization** using ImageNet channel statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).
5. **Random Erasing** [16] with probability $p = 0.3$, which randomly occludes rectangular regions to improve robustness.

Batch-level augmentations. After collation, each mini-batch is randomly assigned one of three modes: CutMix [12] with 25% probability, MixUp [13] with 25% probability ($\alpha = 1.0$ for both), or no augmentation with 50% probability. These mixing strategies generate convex combinations of training pairs, serving as effective regularizers.

Test-Time Augmentation (TTA). During inference, we apply TTA by evaluating each image at three resolutions ($1.0\times$, $1.143\times$, $1.286\times$ of the base size), each with and without horizontal flipping, yielding 6 augmented views. The predicted class probabilities are averaged across all views before selecting the final prediction [3].

2.3. Hyperparameters

We use the following training configuration for our best-performing model:

- **Optimizer:** AdamW [5] with learning rate 1×10^{-4} and weight decay 1×10^{-4} .
- **Scheduler:** CosineAnnealingLR [4] with $T_{\max} = 80$ and $\eta_{\min} = 1 \times 10^{-6}$.
- **Loss:** Cross-entropy with label smoothing $\epsilon = 0.1$ [9].
- **Batch size:** 64.
- **Image size:** 224×224 .
- **Epochs:** 80 (best checkpoint selected by validation accuracy).
- **Dropout:** 0.0 on the penultimate layer (ablation in Sec. 4.1).

3. Results

3.1. Training and Validation Performance

Figure 1 shows the training and validation curves over 80 epochs. The training loss decreases steadily and converges below 0.5, while the training accuracy reaches above 95%. On the validation set, we achieve a best accuracy of **94.00%** at epoch 69. The smooth convergence of both loss and accuracy curves confirms that our augmentation and regularization strategies effectively prevent overfitting despite the relatively small validation set of 300 images.

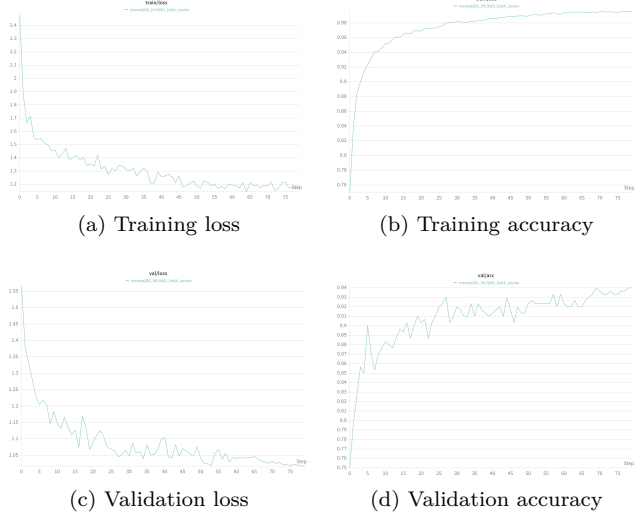


Figure 1. Training and validation curves over 80 epochs for ResNeSt-200 with cosine annealing scheduler, no dropout, and image size 224×224 .

29	cheating	2026-03-31 15:33	658653	111550093	0.96
----	----------	------------------	--------	-----------	------

Figure 2. CodaBench leaderboard screenshot showing our test accuracy of 0.96.

3.2. Competition Performance

With TTA enabled during inference, our best model achieves a test accuracy of **0.96** on the CodaBench competition leaderboard, as shown in Fig. 2. The notable gap between the validation accuracy (94%) and test accuracy (96%) suggests that TTA provides a meaningful boost by aggregating predictions across multiple augmented views, effectively reducing variance in the final predictions.

4. Experiments

4.1. Effect of Dropout Rate

Zhang and Bottou [15] recently demonstrated that applying very large dropout rates (*e.g.*, 90%) on the penultimate layer during fine-tuning can improve out-of-distribution generalization, sometimes exceeding deep ensembles and model soups. Motivated by this finding, we compare dropout rates of 0.0 and 0.9 applied before the final classification layer, with all other settings held constant (ResNeSt-200, cosine scheduler, $\text{lr} = 10^{-4}$, image size 224×224).

As shown in Tab. 1 and Fig. 3, the standard configuration without dropout achieves a slightly higher validation accuracy of 93.00% compared to 92.33% with dropout 0.9. We hypothesize that the extremely aggressive dropout rate, while beneficial for

Dropout Rate	Best Val Acc (%)	Best Epoch
0.0	93.00	71
0.9	92.33	45

Table 1. Validation accuracy comparison between dropout rates of 0.0 and 0.9. Both models use the cosine annealing scheduler with identical hyperparameters.

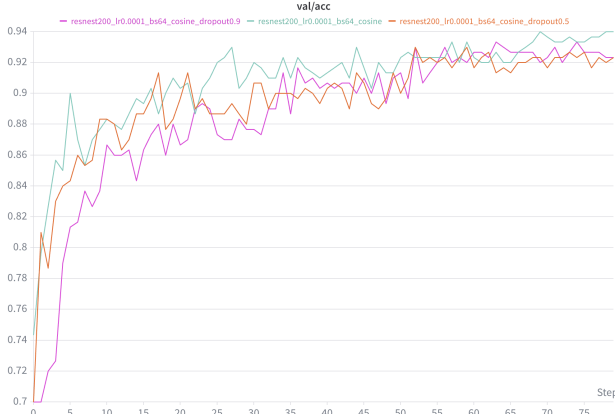


Figure 3. Validation accuracy curves comparing dropout rates of 0.0 and 0.9 over training epochs. The model without dropout achieves higher peak accuracy and more stable convergence.

out-of-distribution scenarios as reported by Zhang and Bottou [15], may be overly restrictive for our in-distribution evaluation setting where the validation and test sets share similar characteristics with the training data. Additionally, the heavy augmentation pipeline (TrivialAugment, CutMix, MixUp, Random Erasing, and label smoothing) already provides substantial regularization, potentially making the additional dropout redundant or even harmful to training convergence.

4.2. Effect of Learning Rate Scheduler

We compare four learning rate scheduling strategies, all using ResNeSt-200 with dropout 0.0, $lr = 10^{-4}$, and image size 224×224 :

- **Cosine Annealing** [4]: Smoothly decays the learning rate following a cosine curve from lr to $\eta_{\min} = 10^{-6}$ over $T_{\max} = 80$ epochs.
- **OneCycleLR** [7]: Ramps the learning rate up to the maximum during the first 10% of training, then anneals it to near zero.
- **ReduceLROnPlateau**: Reduces the learning rate by a factor of 0.1 when the validation loss

Scheduler	Best Val Acc (%)	Best Epoch
Cosine Annealing	94.00	69
OneCycleLR	93.67	76
Plateau	93.33	45
Warm Restarts	93.00	28

Table 2. Validation accuracy comparison across learning rate schedulers. All models use ResNeSt-200, dropout 0.0, and identical hyperparameters. The Cosine Annealing result corresponds to `checkpoints/best.pth`.

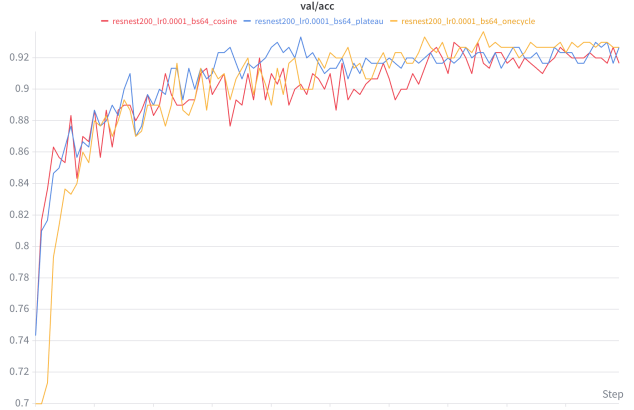


Figure 4. Validation accuracy curves for different learning rate schedulers over training epochs.

plateaus for 5 consecutive epochs.

- **Cosine Annealing with Warm Restarts** [4]: Periodically resets the learning rate with $T_0 = 10$ and $T_{\text{mult}} = 2$, creating increasingly longer annealing cycles.

As shown in Tab. 2 and Fig. 4, Cosine Annealing achieves the highest validation accuracy of 94.00%, followed closely by OneCycleLR at 93.67%. ReduceLROnPlateau and Warm Restarts trail slightly at 93.33% and 93.00%, respectively.

Cosine Annealing benefits from a smooth, monotonic decay that allows the model to gradually settle into a sharp minimum during the later stages of training. OneCycleLR performs comparably, as its warmup phase helps escape poor local minima early on. ReduceLROnPlateau, while adaptive, can be sensitive to the patience hyperparameter and may reduce the learning rate prematurely or too late. Warm Restarts achieves its best accuracy early (epoch 28) but the periodic learning rate resets can destabilize learned features, preventing consistent improvement over longer training.

References

- [1] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009. 1
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. 1
- [3] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012. 1, 2
- [4] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *ICLR*, 2017. 2, 3
- [5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. 2
- [6] Samuel G. Müller and Frank Hutter. TrivialAugment: Tuning-free yet state-of-the-art data augmentation. In *ICCV*, 2021. 1
- [7] Leslie N. Smith and Nicholay Topin. Super-convergence: Very fast training of neural networks using large learning rates. *arXiv preprint arXiv:1708.07120*, 2019. 3
- [8] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. 1
- [9] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, 2016. 1, 2
- [10] Ross Wightman. Pytorch image models. <https://github.com/rwightman/pytorch-image-models>, 2019. 1
- [11] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *CVPR*, 2017. 1
- [12] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *ICCV*, 2019. 1, 2
- [13] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *ICLR*, 2018. 1, 2
- [14] Hang Zhang, Chongruo Wu, Zhongyue Zhang, Yi Zhu, Haibin Lin, Zhi Zhang, Yue Sun, Tong He, Jonas Mueller, R. Manmatha, Mu Li, and Alexander Smola. ResNeSt: Split-attention networks. In *CVPR Workshops*, 2022. 1
- [15] Jianyu Zhang and Léon Bottou. Fine-tuning with very large dropout. *arXiv preprint arXiv:2403.00946*, 2024. 1, 2, 3
- [16] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *AAAI*, 2020. 1